201 Operations for Rank-48 4×4 Matrix Multiplication over Rings Containing 1/2

Greg Sidebottom

Draft — October 22, 2026

Abstract

Rank-48 bilinear schemes multiply 4×4 matrices with 48 multiplications over any ring in which 2 is invertible. Applied recursively, a scheme whose three linear maps use q additions and scalings costs $(1 + q/32)\, n^{\log_4 48} + o(n^{\log_4 48})$ operations, so q fixes the leading constant. The best published count was 204 (Dumas, Pernet, Sedoglavic, Stapleton and Tichavský, 2026), lowered to 202 by R. Patel's value-set search. We give a scheme in the same class whose maps use $55 + 57 + 89 = 201$ operations, leading constant $7 + 9/32$, with no negations, so the count is the same whether or not sign changes are charged. It was found by annealing over the class's symmetry orbit for representatives with a low operation floor and few non-unit coefficients, then running a fast reimplementation of Patel's large-neighbourhood search seeded from the matrices alone, with the output map optimized through its transpose. The scheme is checked by PLinOpt's tools, by an independent exact verifier, by Patel's checker, and by two Lean 4 proofs that it computes AB over every commutative ring in which 2 is invertible. No representative we tried goes below 201, and about twenty reach it exactly. To test whether the programs found so far can be recombined into a 200, we pose that question exactly, as the existence of a closed subset of the union of their signal sets with one element fewer, and encode it for SAT solvers. The answer is no for both input maps at the record point and for all three maps at a second, balanced 201 point. The record's output map is the open case: a 125-vector part of its 137-vector dictionary is refuted, and the remainder is being decided. Encoding acyclicity by a total order instead of by depth levels made these refutations up to 17 times faster.

1 Introduction

Strassen's algorithm applied to 2×2 blocks of 2×2 matrices multiplies 4×4 matrices with 49 multiplications over any ring [22]. In characteristic two the record is 47 [11, 15]. In characteristic zero it is 48: AlphaEvolve found a rank-48 scheme with complex coefficients [18], and Dumas, Pernet and Sedoglavic turned it into one with rational coefficients, valid over any ring containing 1/2 [6, 7]. The rational schemes known today all descend from AlphaEvolve's complex one; Kaporin's independent complex rank-48 scheme [14] has no real equivalent [17].

Used recursively, a scheme's multiplication count fixes the exponent and its additions fix the constant. For an $\langle n_0 \times n_0 \times n_0 : r \rangle$ scheme whose linear maps use q additions and scalings in total,

$$F(n) = r\, F(n/n_0) + q\,(n/n_0)^2 \quad \Longrightarrow \quad F(n) = \Big(1 + \frac{q}{r - n_0^2}\Big) n^{\log_{n_0} r} + o\big(n^{\log_{n_0} r}\big),$$

so at $r = 48$, $n_0 = 4$ the constant is $1 + q/32$. For rank 48, q fell from 315 [6, 9] to 204 [8], and to 202 in October 2026, when Ruchit Patel contributed a 202-operation program to PLinOpt [9] and published the search that found it [19].

Contributions.

- A rank-48 scheme over $\mathbb{Z}[\frac{1}{2}]$ whose linear maps use $55 + 57 + 89 = 201$ operations (constant $7 + 9/32 \approx 7.281$), with no negations; Appendix A lists its programs. It was contributed to PLinOpt (data file `4x4x4_48_201`) and merged on October 22, 2026 (Section 4).
- The method (Section 5): orbit annealing for a representative with a low operation floor and few non-unit coefficients; Patel's value-set large-neighbourhood search, reimplemented in Rust with PLinOpt's objective and seeded from the matrices alone; and a successive-halving sweep over the representatives. Applied to a sparser representative, seeded from the matrix, and run many times over many representatives, Patel's method reaches 201.
- Five independent checks, including two Lean 4 proofs over every commutative ring with 2 invertible (Section 6).
- Evidence that 201 is a plateau for this class under value-set search, and an exact formulation of the question whether the programs found so far can be recombined into a cheaper one, with SAT encodings, verdicts, and the work still in progress (Section 7).

2 Records

Table 1 lists the operation counts for rank-48 schemes without change of basis. With a change of basis the recursive cost drops further, to constants between $5 + 2/3$ and $6 + 13/32$ depending on the encoding [8]; that is a different cost model, and this paper does not use it.

Table 1: Operations (additions and scalings) of rank-48 schemes for $\langle 4, 4, 4\rangle$ over rings containing $1/2$, without change of basis, and the resulting constant of $n^{\log_4 48}$.

operations	constant	split $L + R + P$	source
315	$10 + 27/32$		[6], PLinOpt data [9]
204	$7 + 3/8$		[8]
202	$7 + 5/16$	$56 + 56 + 90$	R. Patel, PLinOpt data [9, 19]
201	$\mathbf{7 + 9/32}$	$55 + 57 + 89$	this paper, PLinOpt data [9]

3 Model

A rank-r scheme for $\langle 4, 4, 4\rangle$ is a triple of matrices $L \in \mathbb{Z}[\frac{1}{2}]^{r\times 16}$, $R \in \mathbb{Z}[\frac{1}{2}]^{r\times 16}$, $P \in \mathbb{Z}[\frac{1}{2}]^{16\times r}$ with

$$\operatorname{vec}(AB) = P\big((L \operatorname{vec} A) \odot (R \operatorname{vec} B)\big),$$

where vec lists entries row by row and $\odot$ is the entrywise product. Product k is $m_k = \langle U_k, A\rangle\langle V_k, B\rangle$ with U_k, V_k the 4×4 matrices in row k of L and R, and $AB = \sum_k m_k W_k$ with W_k in column k of P. The identity is equivalent to the $16^3 = 4096$ Brent equations $\sum_k P_{c,k} L_{k,a} R_{k,b} = [\,a, b \text{ contribute to } c\,]$, a statement over $\mathbb{Q}$. When they hold, the identity holds for matrices over any ring in which 2 is invertible, commutative or not, because each product keeps its A-factor on the left; the scheme therefore recurses on blocks.

Cost. We use PLinOpt's model [9]. A linear straight-line program computes the rows of a matrix M from its inputs by lines $t := u \pm v$ (one operation) and $t := c\,u$ for a constant $c \neq \pm 1$ (one operation, a *scaling*); copies and negations are free. $\mathrm{ops}(M)$ is the fewest operations of such a program, and a scheme costs $\mathrm{ops}(L) + \mathrm{ops}(R) + \mathrm{ops}(P)$. Patel's model also charges each negation; the programs in this paper contain none, so their counts agree in the two models.

Transposition. The output side is handled through its transpose. By the transposition principle [3] a linear program for a matrix without zero rows or columns reverses into one for its transpose with the same scalings, and for $P^{\mathsf{T}} \in \mathbb{Z}[\frac{1}{2}]^{48\times 16}$

$$\mathrm{ops}(P) = \mathrm{ops}(P^{\mathsf{T}}) + (48 - 16) = \mathrm{ops}(P^{\mathsf{T}}) + 32.$$

All three sides are then instances of one problem: compute 48 vectors of $\mathbb{Z}[\frac{1}{2}]^{16}$ from the 16 unit vectors.

Floor. A row that is, up to sign, not a unit vector needs an operation of its own, so $\mathrm{ops}(M) \geq |T(M)|$, where $T(M)$ is the set of distinct such rows up to sign. The *floor* of a scheme is $|T(L)| + |T(R)| + |T(P^{\mathsf{T}})| + 32$.

Symmetry. If $(U_k, V_k, W_k)_k$ is a scheme, so are

$$(X^{\mathsf{T}} U_k Y^{-\mathsf{T}},\ Y^{\mathsf{T}} V_k Z^{-\mathsf{T}},\ X^{-1} W_k Z)_k \qquad \text{and} \qquad (a_k U_k,\ b_k V_k,\ (a_k b_k)^{-1} W_k)_k$$

for invertible X, Y, Z (substitute $A \mapsto XAY^{-1}$, $B \mapsto YBZ^{-1}$) [4] and nonzero scalars a_k, b_k (rescale product k). If $X, Y, Z \in \mathrm{GL}_4(\mathbb{Z}[\frac{1}{2}])$, that is, with determinant $\pm 2^e$, and each a_k, b_k is $\pm$ a power of 2, every coefficient stays in $\mathbb{Z}[\frac{1}{2}]$. All points of an orbit have the same 48 multiplications but different linear maps, and therefore different floors and operation counts; choosing the point is half of the optimization.

4 The 201-operation scheme

Table 2 summarizes the scheme. Its coefficients are small: L and R have entries in $\{0, \pm 1, \pm\frac{1}{2}\}$, with 32 entries $\pm\frac{1}{2}$ each, and P has entries in $\{0, \pm 1, \pm 2\}$, with 8 entries ± 2; 72 coefficients in all are not in $\{0, \pm 1\}$. The floor is $156 = 42 + 44 + 38 + 32$, and the programs spend 45 operations above it, 13 on each input side and 19 on the output side. Each program line is one operation: a binary addition or subtraction, or, four times in all, a halving.

Table 2: The 201-operation scheme, side by side. *Unit rows* are rows that are a single coefficient ± 1 (free); *targets* are the remaining rows, all distinct; *depth* is the longest chain of dependent operations.

side	unit rows	targets	additions	halvings	operations	depth
L	6	42	53	2	55	13
R	4	44	55	2	57	13
P^{T}	10	38	57	0	57	21
P			89	0	$89 = 57 + 32$	21
total			197	4	**201**	

Signs. Negating row k of L (or of R) together with column k of P gives another valid scheme, so the sign of every product's factors is free. The programs as found contain output negations, which PLinOpt does not charge but Patel's model does; we re-emitted them with signs chosen so that no line negates anything. The resulting triple, a point of the same orbit, costs 201 in both models, and it is the one in Appendix A and in PLinOpt.

Other 201s. The count is not tied to one point. About twenty annealed points reach 201, in one of three splits: $55 + 57 + 89$, its mirror image $57 + 55 + 89$, and $56 + 56 + 89$. The last appears at a balanced point with floor $150 = 40 + 40 + 38 + 32$ and 104 non-unit coefficients, the profile of Patel's own representative (Section 7).

5 How the scheme was found

The search ran on October 20–21, 2026, on one Apple M4 Pro machine. Table 3 lists the milestones.

5.1 Choosing a representative

The annealer moves through the orbit by elementary steps on one of X, Y, Z: add c times one row to another with $c \in \{\pm 1, \pm\frac{1}{2}, \pm 2\}$, swap or negate rows, or scale a row by 2 or $\frac{1}{2}$, keeping every coefficient in $\{0, \pm\frac{1}{4}, \pm\frac{1}{2}, \pm 1, \pm 2, \pm 4\}$. It minimizes the floor plus λ times the number of coefficients not in $\{0, \pm 1\}$ (Metropolis acceptance, restarts, a pool of the best distinct points, each re-verified on all 4096 Brent equations). The penalty matters: a low floor with many halves and doubles is expensive to program. The runs found three kinds of points: the profile of the published 204 representative (floor 146 but 136 non-unit coefficients), that of Patel's 202 representative (floor 150, 104), and *sparse* points (floor 156–159, 68–72). The 201 comes from the $\lambda = 0.3$ pool of sparse points.

5.2 Local refinement is not enough

Our first pipeline refined PLinOpt's optimizer output at a fixed point by re-solving small sub-programs. It settled at 238 on the best sparse point and at 253 on Patel's point, whose stored programs cost 202. An exact solver for windows of up to 16 consecutive program lines, with up to four helper values, gained 3 more (235), found nothing on the other sides, and showed the other windows optimal among programs with at most two helpers. So most of the gap was not in how a sub-program computes its values but in which values are computed at all, which is what Patel's method searches over.

5.3 Value-set search

Patel's search [19] identifies a program with its set of signal vectors: the inputs, the targets, and the intermediate values, each a vector of $\mathbb{Z}[\frac{1}{2}]^{16}$. A set is *closed* if its members can be computed one at a time from the inputs, each by one operation on members already computed ($u \pm v$, $2u$, $u/2$, or a rescaling of a proportional member); a closed set yields a program with one operation per non-input member, so the cost is the set's size minus 16. Each step of the search is a large-neighbourhood move [20]: it proposes one to eight new vectors from six families (residuals of targets against members, coordinated packages of residuals, sparse pivots, residuals among members, halvings and doublings, and broad random combinations), adds them, and then deletes members in random order while the set stays closed. Steps that lower the cost are accepted, steps that keep it are accepted

unless the new set is in a tabu list of recent sets, and steps that raise it by one are accepted with a small probability.

5.4 A faster implementation, and seeds from the matrix

Patel's Python spends most of a step in a sign planner for his cost model, which charges negations; PLinOpt's model does not need it. We reimplemented the search in Rust (`src/bin/lns48.rs`): vectors in fixed point over 1/64, closure and deletion on bitsets, PLinOpt's objective, and an exact re-verification of every program written. It runs about 80 steps per second per thread, 70 times the original.

The second change was the seed. Every run seeded with programs derived from PLinOpt's optimizer, Python or Rust, converged to 56 on the sparse point's L; seeded instead with the naive program that computes each row of the matrix as a chain of additions (pruned by the same deletion before the search starts), the search reached 55 in twenty seconds. The seed decides the basin, so we replaced long runs by many short restarts from matrix-only seeds: twelve seeds of three minutes per side reached $202 = 55 + 58 + 89$ on the sparse point. The output side is always searched as P^{T}, a 48×16 map like L and R, and transposed back with PLinOpt's `transpozer`.

5.5 The sweep

The last step was a successive-halving sweep [13] over 90 annealed points: 30 seconds per side for every point, then 120 seconds for the best 20, then 600 seconds with two seeds for the best 6. Two points reached 201 in the second stage, and all six finalists ended at exactly 201.

Table 3: Milestones of the search, October 20–21, 2026. Counts are PLinOpt's (additions and scalings).

total	$L + R + P$	step
238	$70 + 67 + 101$	sparse point, PLinOpt optimizer plus pair-level re-solving
235	$67 + 67 + 101$	exact window search on L
205	$56 + 58 + 91$	Patel's search [19] on the same point, P through P^{T}
204	$56 + 58 + 90$	Rust reimplementation
203	$55 + 58 + 90$	seeded from the matrix alone
202	$55 + 58 + 89$	twelve short restarts per side (ties Patel's 202)
201	$55 + 57 + 89$	sweep over 90 annealed points

6 Verification

The triple in Appendix A passes five independent checks.

1. PLinOpt's `SLPchecker` accepts each program against its matrix and reports $53 + 2$, $55 + 2$ and $89 + 0$ additions plus scalings.

2. PLinOpt's `MMchecker` accepts the three matrices as a matrix multiplication scheme, in rational mode, on random inputs (five runs, three more with 40-bit values).

3. Our verifier `matmul/dps48/slp48check.py` evaluates the programs exactly over $\mathbb{Q}$ and checks all 4096 Brent equations (0 failures), counting operations in both cost models.

4. Patel's own checker [19] reports 201 linear operations, 4096 tensor identities, and a well-formed program DAG.

5. Two Lean 4 [5] proofs with Mathlib [16], checked by Lean's kernel. `Matmul48_201.correct` (`proof/Mm/Correct48_201.lean`) takes the three programs line by line as hypotheses, 201 operation wires and 48 products with each halving written $u{\cdot}h$, and proves that the 16 outputs are the entries of AB. `rank48_op201_scheme_eq_mul` (`proof/Mm/Ring_4x4_48_201.lean`) states the result in Mathlib's own terms:

   ```
   theorem rank48_op201_scheme_eq_mul (h : R) (h2 : 2 * h = 1)
       (A B : Matrix (Fin 4) (Fin 4) R) :
       rank48_op201_scheme h A B = A * B
   ```

   for every commutative ring `R`, where the scheme is a function of 201 commented `let` bindings and 48 products. Each entry is closed by `linear_combination` $e{\cdot}(2h-1)$, with the polynomial e computed by the generator through exact division by $2h-1$. The proofs check in about 15 and 90 seconds and use only Lean's three standard axioms.

Validity over non-commutative rings follows from check 3, since the Brent equations are exactly the condition for the bilinear identity with non-commuting entries.

7 Can 201 be lowered? Work in progress

7.1 A plateau

No route we tried goes below 201:

- about twenty distinct annealed points with near-minimal penalized floor reach exactly 201, in one of the three splits above, and none goes lower; of the 19 such points within two orbit moves of the record point, three reach 201 and the rest 203;
- other profiles, from annealer variants that weight the three sides differently: points with side floors 42/42/42 reach $L + R = 110$ but $P = 93$ (203), and random orbit walks that ignore the floor cost 212 or more;
- longer and wider searches at the record point: hot annealing (330,000 steps), proposal batches of 16 and 24 vectors with eight deletion orders, Metropolis acceptance at two temperatures, searching P directly as a 16×48 map, and replacing the random deletion by an exact minimum closed subset (a mixed-integer program solved by HiGHS [12] over the set plus 60 proposals, about 2000 exact steps): R and P^{T} stay at 57 and P at 89 throughout.

Two regularities hold at the floor-150 and floor-156 points: $\mathrm{ops}(P^{\mathsf{T}}) = |T(P^{\mathsf{T}})| + 19$, so P costs $|T(P^{\mathsf{T}})| + 51$ (also at the 42/42/42 points), and $\mathrm{ops}(L) + \mathrm{ops}(R) = 112$.

7.2 Recombination as an exact question

Independent runs end in different programs with the same cost. Could a mixture of them do better? Let the *dictionary* D be the union of the signal sets of a collection of programs for one side, and let a *production* of $v \in D$ be one operation that computes v from other members, found by enumerating all pairs of members (sums, differences, proportional rescalings). The question

is there a closed set S with inputs $\subseteq S \subseteq D$, all targets in S, and at most c non-input members?

asks whether the side can be programmed in c operations using only vectors that some program in the collection computes. Because the productions of a subset are determined by its own members, a negative answer for a dictionary is also a negative answer for every subset of it. The question can be enlarged by adding vectors to D that no program used: search proposals, or a *structured* family, all sub-vectors of at most three terms of each target (with the target's coefficients) together with all pairwise sums and differences of targets.

7.3 Encodings

The first formulation was a mixed-integer program with a selection variable x_v per vector, a variable y_p per production, and continuous levels with big-M constraints for acyclicity, solved by HiGHS [12]. Its feasibility form proves small cases in a minute, but its relaxation is weak: four runs on dictionaries of 86–137 vectors reached their six-hour limits undecided.

The SAT encoding (`lns48 --merge ... --cap c --cnf`) uses the same x_v and y_p with clauses

$$x_t \ (t \text{ a target}), \qquad y_p \to x_u \ (u \text{ an operand of } p), \qquad x_v \to \bigvee_{p \text{ produces } v} y_p,$$

a sequential counter for $\sum x_v \le c$ [21], and three strengthenings that are sound because any solution can be reduced to one that satisfies them: $y_p \to x_v$, at most one production per vector, and every selected non-target is an operand of a selected production. Acyclicity is encoded in one of two ways.

- *Levels*: $g_{v,j}$ means that v has depth at least j, for $j \le \ell$, with clauses that place each selected production's result above its operands. This is complete only if ℓ is at least the depth of some optimal program; $\ell = c$ always is.
- *Order*: one variable per pair of non-input vectors and two transitivity clauses per triple encode a total order, and each selected production puts its operands before its result. There is no depth bound.

A satisfying assignment is decoded into a program and re-verified. We validated the encoding on questions with known answers: on the record point's L dictionary it answers no at 54 and yes at 55, decoding to a correct 55-operation program; and the structured dictionary for R at $c = 57$ is satisfiable, with the record's own program as the model. The solvers are CaDiCaL 3.0.0 [1] and kissat 4.0.4 [2].

7.4 Verdicts

Table 4 lists the verdicts so far. At the record point no recombination of the programs found lowers L or R, for R even with 200 added proposals; at the balanced point none lowers any of the three sides, so $201 = 56 + 56 + 89$ is final there for this question. The record's P^{T} is the open case. Its programs are deep, with chains of 21 dependent operations against 13 on L and R, because rows of P^{T} are built from one another in long chains. The level encoding pays for that depth. Bounded depth is easy: no 56-operation program of depth at most 6 exists (0.09 seconds), nor of depth at most 10 (748 seconds). With the full depth the question stayed open for eight hours under both solvers and every split we tried. The order encoding removes the depth variables altogether, and it refutes the balanced point's P^{T} in 306 seconds, against 88 minutes with levels.

On the record's P^{T} dictionary, built from 32 programs, the order encoding refutes the parts contributed by the first 22, 24 and 28 programs in 33, 406 and 5000 seconds. The time grows about twelvefold per 14 vectors, which puts the whole 137-vector question near twelve hours. The refuted parts give a shortcut. Since the first 125 vectors admit no closed set of 56, any solution over all 137 must select at least one of the 12 vectors added by the last four programs. We split that condition into 12 disjoint cases (vector i selected, vectors $1, \ldots, i-1$ not), each a smaller question with one vector forced in; these are running now, beside whole-instance runs as a baseline.

Table 4: Recombination questions. *Vectors* counts the dictionary including the 16 inputs; c is the operation budget, one below the side's best program. Times are wall-clock on a machine running other solver jobs in parallel.

point	side	vectors	c	verdict	how
record	L	82	54	no	MIP 90 s; SAT 0.4 s
record	R	81	56	no	MIP 51 s
record	R	281	56	no	+200 proposals; CaDiCaL ≈30 s
record	R	2862	56	open	structured; kissat 8 h, $\ell = 24$
balanced	L	86	55	no	CaDiCaL ≈30 s; MIP open at 6 h
balanced	R	92	55	no	CaDiCaL ≈30 s; MIP open at 6 h
balanced	P^{T}	112	56	no	order 306 s; levels 88 min (kissat)
record	P^{T}	97	56	no	order 33 s; levels 98 s
record	P^{T}	111	56	no	order 406 s
record	P^{T}	125	56	no	order 5000 s
record	P^{T}	137	56	running	split into 12 cases
record	P^{T}	337	56	open	+200 proposals; 8 h, levels
record	P^{T}	3295	56	open	structured; 8 h, $\ell = 24$

7.5 What a no means, and what comes next

A refutation is a statement about a dictionary: no program built only from those vectors is cheaper. A 200 could still exist that computes a vector no search has produced, and these verdicts are unchecked solver output so far. The next steps are:

1. finish the record's P^{T} question, and if it is refuted, enlarge the dictionary by proposals and structured vectors, using the same incremental split;

2. certify every refutation used in a claim with a DRAT proof checked by an independent checker [23];

3. try cutting-planes pseudo-Boolean solving [10], which reasons about the budget $\sum x_v \le c$ directly instead of through a counter's clauses; the converter to OPB is written;

4. seek lower bounds that apply to every program, not only to a dictionary: the floor of the record point is 156, and a closure-level bound after reduction modulo 3, where halvings become free sign changes, raises it only to 162 (for programs whose scalings are powers of 2), far below 201;

5. look beyond this class. Every operation count in Table 1 belongs to the class descending from AlphaEvolve's scheme, and a second rank-48 class over $\mathbb{Q}$ would be a new starting point.

8 Data and code

Everything is in the public repository https://github.com/gsidebottom/logic; paths below are relative to it.

- The scheme: `matmul/dps48/cand48/p005_201n_{L,R,P}.{slp,sms}`, with `p005_201_Pt.slp`, the 57-operation program for P^{T}. The same files are in PLinOpt as `data/4x4x4_48_201_{L,R,P}.{slp,sms}`.
- Orbit annealing: `matmul/dps48/orbit48.py`.
- Value-set search, recombination, and the SAT and MIP encodings: `src/bin/lns48.rs`.
- Sweep and restarts: `matmul/dps48/patel/sweep.py` and `restarts.sh`.
- Sign normalization and the exact verifier: `matmul/dps48/signnorm48.py` and `slp48check.py`.
- Case splits and OPB conversion: `matmul/dps48/cube.py` and `cnf2opb.py`.
- The Lean proofs: `proof/Mm/Correct48_201.lean` and `proof/Mm/Ring_4x4_48_201.lean`. One of them runs alone with

  ```
  cd proof && lake build Mm.Ring_4x4_48_201
  ```

- Appendix A: generated by `matmul/dps48/mk_paper_appendix48.py`, which re-checks the Brent equations first.

Acknowledgments

This work was carried out in an extended interactive collaboration with Claude (Anthropic), which implemented the search and verification tools and much of this text under the author's direction and review. Every claim is checked mechanically by the tools above; the results rest on those checks rather than on trust in either the author or the tools. The method that found the scheme is Ruchit Patel's, whose search code is public domain.

References

[1] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks, F. Pollitt, *CaDiCaL 2.0*, Proc. CAV 2024, LNCS 14681, 133–152.

[2] A. Biere, M. Fleury et al., *Kissat SAT solver*, https://github.com/arminbiere/kissat (version 4.0.4).

[3] P. Bürgisser, M. Clausen, M. A. Shokrollahi, *Algebraic Complexity Theory*, Springer, 1997.

[4] H. F. de Groote, *On varieties of optimal algorithms for the computation of bilinear mappings I. The isotropy group of a bilinear mapping*, Theoret. Comput. Sci. 7 (1978) 1–24.

[5] L. de Moura, S. Ullrich, *The Lean 4 theorem prover and programming language*, Proc. CADE-28 (2021), LNCS 12699, 625–635.

[6] J.-G. Dumas, C. Pernet, A. Sedoglavic, *A non-commutative algorithm for multiplying* 4×4 *matrices using* 48 *non-complex multiplications*, arXiv:2506.13242 (2025).

[7] J.-G. Dumas, C. Pernet, A. Sedoglavic, *A more accurate rational non-commutative algorithm for multiplying* 4×4 *matrices using* 48 *multiplications*, arXiv:2603.18699 (2026).

[8] J.-G. Dumas, C. Pernet, A. Sedoglavic, J. Stapleton, P. Tichavský, *Fast matrix multiplication via recursive* $\langle 4 \times 4 \times 4 : 48 \rangle$ *algorithms into practice*, arXiv:2609.12027 (2026).

[9] J.-G. Dumas, B. Grenet, C. Pernet, A. Sedoglavic, *PLinOpt, a collection of C++ routines handling linear & bilinear programs*, `https://github.com/jgdumas/plinopt` (data files `4x4x4_48_202` and `4x4x4_48_201`, October 2026).

[10] J. Elffers, J. Nordström, *Divide and conquer: towards faster pseudo-Boolean solving*, Proc. IJCAI 2018, 1291–1299.

[11] A. Fawzi et al., *Discovering faster matrix multiplication algorithms with reinforcement learning*, Nature 610 (2022) 47–53.

[12] Q. Huangfu, J. A. J. Hall, *Parallelizing the dual revised simplex method*, Math. Program. Comput. 10 (2018) 119–142.

[13] K. Jamieson, A. Talwalkar, *Non-stochastic best arm identification and hyperparameter optimization*, Proc. AISTATS 2016, 240–248.

[14] I. E. Kaporin, *Finding complex-valued solutions of Brent equations using nonlinear least squares*, Comput. Math. Math. Phys. 64 (2024) 1881–1891.

[15] M. Kauers, J. Moosbauer, *Flip graphs for matrix multiplication*, Proc. ISSAC 2023; arXiv:2212.01175.

[16] The mathlib Community, *The Lean mathematical library*, Proc. CPP 2020, 367–381.

[17] Y. Moran, O. Schwartz, S. Yuan, *Complex to rational fast matrix multiplication*, arXiv:2602.13171 (2026).

[18] A. Novikov et al., *AlphaEvolve: a coding agent for scientific and algorithmic discovery*, arXiv:2506.13131 (2025).

[19] R. Patel, *matmul-4x4-48-202*, search code and programs, `https://github.com/rp440/matmul-4x4-48-202` (October 2026, CC0).

[20] P. Shaw, *Using constraint programming and local search methods to solve vehicle routing problems*, Proc. CP 1998, LNCS 1520, 417–431.

[21] C. Sinz, *Towards an optimal CNF encoding of Boolean cardinality constraints*, Proc. CP 2005, LNCS 3709, 827–831.

[22] V. Strassen, *Gaussian elimination is not optimal*, Numer. Math. 13 (1969) 354–356.

[23] N. Wetzler, M. J. H. Heule, W. A. Hunt Jr., *DRAT-trim: efficient checking and trimming using expressive clausal proofs*, Proc. SAT 2014, LNCS 8561, 422–429.

A The 201-operation programs

The three programs below are the sign-normalized triple stored as `matmul/dps48/cand48/p005_201n_{L,R,P}.slp` and in PLinOpt as `data/4x4x4_48_201_{L,R,P}.slp`, in PLinOpt syntax. In L and R the input `i`$(4i{+}j)$ is a_{ij} (resp. b_{ij}), indices from 0, and output `o`k is the factor of product m_k ($k = 0, \ldots, 47$); in P the input `i`k is m_k and output `o`$(4i{+}j)$ is c_{ij}. Each line is one operation (a binary addition or subtraction, or a halving `/2`) or a free copy `o`k`:=`x. The products are $m_k = \ell_k\, r_k$ with ℓ_k the k-th output of L (left factor) and r_k that of R. The generator re-evaluates the programs over $\mathbb{Q}$ and checks all 4096 Brent equations before writing this appendix.

Left factors L: 53 additions + 2 halvings = 55 operations

```
t1:=i9-i10;
t2:=i12-i15;
t3:=i6-i5;
t4:=i7+t3;
t5:=i0-i1;
t6:=i2+i3;
t7:=i4-i5;
t8:=i11-i8;
t9:=i14+i15;
t10:=t9-i13;
t11:=i3-i7;
t12:=i9-i13;
t13:=i2-i10;
t14:=t12/2;
t15:=t14+t9;
t16:=t14-t7;
t17:=t14-i10;
t18:=t11/2;
t19:=t18-t8;
t20:=t13-i14;
t21:=t18-t6;
t22:=t13-i6;
t23:=t19-i8;
t24:=i9-t14;
t25:=i4+t16;
t26:=t15+t21;
t27:=t5+t24;
t28:=t4+t18;
t29:=t26-t28;
t30:=t22+t25;
t31:=t17-t26;
t32:=t30+t26;
t33:=i0-t27;
t34:=t31-t20;
t35:=t29-t25;
t36:=t32+t23;
t37:=t31+t23;
t38:=t29+t33;
t39:=t32+t34;
t40:=i12+t34;
t41:=t30-t33;
t42:=t16+t40;
t43:=t27-t42;
t44:=t43+t34;
t45:=t19-t42;
t46:=t43-t40;
t47:=t45+t25;
t48:=t46-t45;
t49:=t14-t45;
t50:=t2+t46;
t51:=t48+t42;
t52:=t48+t33;
t53:=t18-t51;
t54:=t48+t24;
t55:=t51+t23;
o0:=t1;
o1:=t2;
o2:=t4;
o3:=t51;
o4:=t35;
o5:=t52;
o6:=t36;
o7:=i4;
o8:=t15;
o9:=i2;
o10:=t47;
o11:=t27;
o12:=t6;
o13:=t53;
o14:=t28;
o15:=i0;
o16:=t43;
o17:=t54;
o18:=t48;
o19:=t49;
o20:=t8;
o21:=t16;
o22:=t37;
o23:=t9;
o24:=t29;
o25:=i10;
o26:=t44;
o27:=t17;
o28:=t10;
o29:=t55;
o30:=i12;
o31:=t45;
o32:=t38;
o33:=t19;
o34:=t30;
o35:=t7;
o36:=t20;
o37:=t31;
o38:=t39;
o39:=t40;
o40:=t5;
o41:=i8;
o42:=t3;
o43:=t21;
o44:=t41;
o45:=t22;
o46:=t50;
o47:=t32;
```

Right factors R: 55 additions + 2 halvings = 57 operations

```
t1:=i1+i5;
t2:=i9+t1;
t3:=i0+i4;
t4:=i14-i10;
t5:=i2+i6;
t6:=i0+i12;
t7:=i3+i15;
t8:=i3+i7;
t9:=t6-i8;
t10:=i11+t8;
t11:=i9-i13;
t12:=i1-i3;
t13:=i8+i10;
t14:=i14+i15;
t15:=t14-i13;
t16:=t14-i12;
t17:=t11-i1;
t18:=t12/2;
t19:=t13/2;
t20:=i8-t19;
t21:=t7+t18;
t22:=i6+t18;
t23:=t19-i9;
t24:=t15-t23;
t25:=t17+t18;
t26:=t23-t1;
t27:=t10-t20;
t28:=t6-t19;
t29:=i14-t19;
t30:=t22-t5;
t31:=t8-t27;
t32:=t28-t21;
t33:=t29-t32;
t34:=t24+t32;
t35:=t30-t33;
t36:=t24+t31;
t37:=t25+t32;
t38:=t23+t37;
t39:=t34-t30;
t40:=t16-t33;
t41:=t37-t31;
t42:=t34-t40;
t43:=t3-t40;
t44:=t43-t27;
t45:=t44-t22;
t46:=t26+t44;
t47:=t43+t46;
t48:=t45-t31;
t49:=t47-t45;
t50:=t46+t40;
t51:=t47-i4;
t52:=t49-t44;
t53:=t49+t19;
t54:=t45+t20;
t55:=t52+t18;
t56:=t52-t30;
t57:=t49-t23;
o0:=t48;
o1:=t42;
o2:=t15;
o3:=t21;
o4:=t2;
o5:=t3;
o6:=t38;
o7:=t56;
o8:=t49;
o9:=t4;
o10:=t50;
o11:=t22;
o12:=i14;
o13:=i15;
o14:=t24;
o15:=t5;
o16:=t25;
o17:=i4;
o18:=t43;
o19:=t51;
o20:=t16;
o21:=t52;
o22:=t6;
```

```
o23:=t53;
o24:=t26;
o25:=t54;
o26:=t17;
o27:=t45;
o28:=t57;
o29:=t7;
o30:=t39;
o31:=t46;
o32:=t1;
o33:=t33;
o34:=t27;
o35:=t55;
o36:=t9;
o37:=t28;
o38:=t41;
o39:=t34;
o40:=i6;
o41:=t35;
o42:=t36;
o43:=t29;
o44:=t8;
o45:=t10;
o46:=t11;
o47:=t37;
```

Output map P: 89 additions = 89 operations

```
t1:=i43+i12;
t2:=i9-i12;
t3:=t2+i40;
t4:=i11-t3;
t5:=i15-t3;
t6:=i3+i13;
t7:=i29+i13;
t8:=t4+i31;
t9:=t8+i19;
t10:=i21+i35;
t11:=i27+t6;
t12:=t11+i25;
t13:=t2-t12;
t14:=t9+t13;
t15:=t10-t14;
t16:=t12+t15;
t17:=i17+t16;
t18:=t17-i18;
t19:=i20-t18;
t20:=i25+i0;
t21:=t18-t20;
t22:=i33+t19;
t23:=t1-t15;
t24:=i8-t14;
t25:=t24-i23;
t26:=i16+t25;
t27:=i34+t22;
t28:=t25-t22;
t29:=t28+t23;
t30:=t23-i47;
t31:=t13+t29;
t32:=i46-t26;
t33:=i26-t26;
t34:=t18+t32;
t35:=t6+t1;
t36:=t35+t31;
t37:=i37+t4;
t38:=t37+t31;
t39:=i39+t36;
t40:=i24+t36;
t41:=i28-i23;
t42:=t41+t34;
t43:=t34-i1;
t44:=t43+t39;
t45:=t27-i45;
t46:=i4+t40;
t47:=t44-t17;
t48:=t47+i5;
t49:=i10-t8;
t50:=t49+t45;
t51:=t50+t44;
t52:=t51-t46;
t53:=i14-t4;
t54:=t53+t29;
t55:=t54+t32;
t56:=i21+i7;
t57:=t56-t46;
t58:=t57-t45;
t59:=t55-i42;
t60:=t10-i2;
t61:=t60+t55;
t62:=t59+t45;
t63:=i36-t16;
t64:=t63-t38;
t65:=t38-i22;
t66:=t44+t64;
t67:=t33-t64;
t68:=t42+t33;
t69:=t65+t7;
t70:=t42-t61;
t71:=t61-t46;
t72:=t59+t21;
t73:=t21-t7;
t74:=i32+t40;
t75:=t74+t32;
t76:=t75+t70;
t77:=i30-t39;
t78:=t77+t67;
t79:=t27-i44;
t80:=t79+t72;
t81:=t19-i41;
t82:=t81+t69;
t83:=i38-t67;
t84:=t83+t72;
t85:=t84+t30;
t86:=i6-t9;
t87:=t86+t70;
t88:=t87+t69;
t89:=t88+t30;
o0:=t48;
o1:=t76;
o2:=t5;
o3:=t80;
o4:=t52;
o5:=t71;
o6:=t58;
o7:=t62;
o8:=t65;
o9:=t89;
o10:=t82;
o11:=t73;
o12:=t66;
o13:=t68;
o14:=t78;
o15:=t85;
```